

Series — Arithmetic and Geometric

A 1.5-2 hour lecture on finite series, formulas, derivations, applications and Python verification.

- Recommended teaching duration: approximately 1.5-2 hours including board work, worked examples, Python demonstrations and student discussion.
- Teaching approach: concept → notation → worked example → CS interpretation → Python verification → student practice.

Lecture Roadmap

- Concept introduction and terminology
- Detailed definitions and notation
- Worked mathematical examples
- CS case studies with complete solutions
- Python implementation and verification
- Extended question bank
- 2-3 unsolved student practice questions at the end

1. Sequence vs Series

A sequence lists terms; a series adds terms. For example 2,4,6,8 is a sequence while $2+4+6+8$ is a series.

2. Arithmetic Series

For first term a , common difference d and n terms: $S_n = n/2[2a + (n-1)d]$. If the last term l is known, $S_n = n(a+l)/2$.

The formula can be motivated by pairing first and last terms.

3. Geometric Series

For first term a , ratio r and n terms, $r \neq 1$: $S_n = a(r^n - 1)/(r - 1)$.

4. Choosing a Model

Use an arithmetic series when successive amounts change by a constant. Use a geometric series when successive amounts are multiplied by a constant ratio.

5. CS Application — Cumulative Work

Series can represent total requests, total storage, total memory allocation, total test cases, or total cost across stages.

6. Case Study — API Requests

A test system sends 100 requests in minute 1 and adds 50 requests each minute for 20 minutes.

Solution: arithmetic series with $a=100, d=50, n=20$. $S_{20}=20/2[200+950]=11,500$ requests.

7. Case Study — Backup Storage

A backup stores 5 GB in stage 1 and doubles storage each stage for 8 stages.

Solution: geometric series: $5(2^8-1)/(2-1)=1275$ GB total.

8. Comparing Term and Total

The final term is not the same as cumulative total. In the backup case, stage 8 is 640 GB, but total storage across eight stages is 1275 GB.

9. Python Verification

Generate the terms and use `sum()`. Compare with the closed-form formula. This is a useful habit for detecting arithmetic errors.

10. Extension — Infinite Geometric Idea

When $|r|<1$, an infinite geometric series approaches a finite sum $a/(1-r)$. This can be introduced conceptually even if the main focus is finite series.

Python Laboratory

Python Activity 1

```
a=100; d=50; n=20
terms=[a+(i-1)*d for i in range(1,n+1)]
formula=n/2*(2*a+(n-1)*d)
print(sum(terms), formula)
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 2

```
a=5; r=2; n=8
terms=[a*r**(i-1) for i in range(1,n+1)]
formula=a*(r**n-1)/(r-1)
print(sum(terms), formula)
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 3

```
# Infinite geometric series when |r| < 1
a=10; r=0.5
print(a/(1-r)) # 20
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Extended Question Bank — Instructor-Led Practice

The following questions are designed to provide enough board/classroom practice for a 1.5-2 hour session. They cover the same topic areas as the relevant VU MTH202 sequence, but are written as fresh exercises rather than reproducing the handout wording.

1. Find the sum of the first n terms of an arithmetic series.
2. Find the sum of a finite geometric series.
3. Use the last-term form $S = n(a+l)/2$.
4. Determine whether a workload should be modeled by arithmetic or geometric growth.
5. Find the total API requests in a staged workload.
6. Find cumulative backup storage.
7. Verify an arithmetic series with Python.
8. Verify a geometric series with Python.
9. Explain why the final term and total series answer different questions.
10. Calculate an infinite geometric sum for $|r| < 1$.
11. Write Python functions `arithmetic_sum` and `geometric_sum`.
12. Design a CS scenario that naturally produces an arithmetic series.

End-of-Lecture Student Practice — No Solutions

1. Model a cumulative workload as an arithmetic series and solve it.
2. Model a storage process as a geometric series and solve it.
3. Verify one result using Python.

Quick Recap

Series model cumulative totals and resource accumulation.